```
 function register ( ) public payable{

     address AirlineA = msg.sender;
     membership[AirlineA] = 1;
     escrow[AirlineA] = msg.value;
 }

function unregister (address payable AirlineZ) onlyChairperson public {
     membership[AirlineZ]=0;
     //return escrow to leaving airline:other conditions may be verified
     AirlineZ.transfer(escrow[AirlineZ]);
     escrow[AirlineZ] = 0;
 }

 function ASKrequest (uint reqID, uint flightID, uint numSeats,
           uint custID, address toAirline) onlyMember public{
     /*if(membership[toAirline]!=1){
         revert();}  */
     require(membership[toAirline] == 1);
     reqs[msg.sender] = reqStruc(reqID, flightID, numSeats,
                       custID, toAirline);                    ❻

 }

 function  ASKresponse (uint reqID, bool success, address fromAirline)
                                  onlyMember public{

     if(membership[fromAirline]!=1){
         revert();
     }

     reps[msg.sender].status=success;                         ❻
     reps[msg.sender].fromAirline = fromAirline;              ❻
     reps[msg.sender].reqID = reqID;                          ❻
 }

 function settlePayment  (uint reqID, address payable toAirline,
                       uint numSeats) onlyMember payable public{
     //before calling this, it will update ASK view table
     address fromAirline = msg.sender;

     //this is the consortium account transfer you want to do
     //assume the cost of 1 ETH for each seat
     // computations are in wei

     escrow[toAirline] = escrow[toAirline] +
                       numSeats*1000000000000000000;
     escrow[fromAirline] = escrow[fromAirline] -
                       numSeats*1000000000000000000;

     settledReqID[fromAirline] = reqID;                       ❼
 }

 function replenishEscrow() payable public
 {
```